# Laboratorio 6 - Visión Por Computadora

**Autores:**

- **Nelson García**
- **Joaquín Puente**
- **Diego Linares**

# Task 1

1. **Un desarrollador junior de su equipo sugiere: "Para detectar con mayor precisión las texturas de las hojas enfermas, deberíamos construir una red secuencial clásica (tipo VGG) pero de 150 capas. Más profundo siempre es mejor". Como líder técnico, explíquele argumentativamente por qué esta red fracasará estrepitosamente en el entrenamiento (mencionando el fenómeno de degradación y el desvanecimiento del gradiente). Luego, justifique cómo la adición estructural de las conexiones residuales ($F(x) + x$) de ResNet rescata el proyecto, haciendo viable entrenar redes ultra-profundas sin colapsar.**

**R//**

**Es cierto que VGG mostró que aumentar la profundidad puede mejorar la capacidad de representación, pero existe un límite práctico y matemático. En redes muy profundas aparecen dos problemas centrales. El primero es el desvanecimiento del gradiente, donde durante la retropropagación los gradientes se vuelven cada vez más pequeños al atravesar muchas capas, por lo que las capas tempranas casi dejan de aprender. El segundo es la degradación de la precisión, que significa que al seguir agregando capas el error de entrenamiento aumenta en vez de disminuir.**

**En una arquitectura puramente secuencial, cada capa debe aprender una transformación completa sobre la salida anterior. En teoría esto parece más expresivo, pero en la práctica vuelve muy inestable la optimización. Si una parte de la red adicional no aporta valor, el modelo no tiene un mecanismo simple para comportarse como identidad y dejar pasar la información sin distorsionarla. Por eso una VGG extrema de 150 capas tendería a entrenar lentamente, estancarse o incluso rendir peor que una versión mucho más corta, aun teniendo más capacidad nominal.**

**ResNet es la clave en este caso, ya que, en lugar de obligar a la red a aproximar directamente una transformación completa H(x), el bloque residual aprende solo el residuo F(x), de modo que la salida final se expresa como H(x) = F(x) + x. En la retropropagación aparece un término adicional igual a 1 en la derivada con respecto a la entrada del bloque, lo cual crea una ruta directa para el flujo del gradiente y reduce el desvanecimiento. En otras palabras, aunque la rama convolucional aprenda poco o se vuelva difícil de optimizar, la señal todavía puede propagarse por el atajo de identidad.**

**Entonces, no conviene apostar el proyecto AgriTech a una VGG de 150 capas solo por ser más profunda. Esa decisión elevaría el riesgo de entrenamiento fallido, pérdida de tiempo de GPU y retrasos de entrega. En cambio, una ResNet convierte la profundidad en una ventaja utilizable, porque preserva el flujo de información y de gradientes, reduce el colapso de optimización y vuelve factible entrenar modelos ultra profundos para capturar texturas finas en hojas enfermas sin destruir la estabilidad del proceso.**

1. **Las enfermedades en las hojas de mango son visualmente heterogéneas: La Antracnosis se presenta como puntos negros diminutos, mientras que el Moho Polvoriento cubre áreas enormes de la hoja. Analice cómo la topología en paralelo del módulo Inception (usando filtros 3x3 y 5x5 simultáneamente) es ideal para este problema biológico en particular. Además, desde una perspectiva de costos de infraestructura (uso de GPUs en AWS o Google Cloud), explique cómo la inserción estratégica de convoluciones de 1x1 evita la explosión de la dimensionalidad y salva el presupuesto mensual de la startup.**

**R//**

**El problema biológico de las hojas de mango exige una arquitectura que vea patrones en múltiples escalas espaciales al mismo tiempo. Para los puntos negros de la Antracnosis, se require filtros pequeños capaces de capturar detalles locales y texturas finas. En cambio, el Moho Polvoriento cubre regiones más amplias de la**

hoja, por lo que conviene usar filtros mayores que integren contexto espacial más global.

La topología en paralelo de Inception evita permite procesar la misma entrada por varias ramas simultáneas, y luego concatena los mapas de características obtenidos. Así, la red aprende de manera conjunta rasgos pequeños y patrones extensos, lo que viene siendo las enfermedades en los árboles de mangos.

Sin embargo, la multiescala de este modelo tiene un peligro operativo. Si cada rama procesa todos los canales de entrada con convoluciones costosas y luego se concatenan todas las salidas, la dimensionalidad puede crecer rápidamente. Esto incrementa parámetros, memoria intermedia y FLOPs, lo cual se traduce en tiempos de entrenamiento más altos y facturas más caras en servicios de nube.

Lo mejor sería insertar convoluciones de 1x1 antes de otras ramas, como 3x3 o 5x5. Matemáticamente, estas capas actúan como proyectores lineales sobre la dimensión de canales. No cambian la resolución espacial, pero sí reducen la cantidad de canales que entrará a las convoluciones posteriores. Reducir canales antes de aplicar filtros más grandes disminuye de manera importante las operaciones requeridas.

1. El cliente final (el agricultor) usará la aplicación en un teléfono Android de gama baja en medio del campo, sin conexión a internet. Sabemos que MobileNet logra esta eficiencia gracias a la Depthwise Separable Convolution. Describa brevemente cómo esta convolución divide el trabajo (filtrado espacial vs. combinación de canales). Sin embargo, en ingeniería no hay soluciones mágicas, todo tiene un precio: ¿Qué capacidad matemática o expresividad de la red se está sacrificando al separar estos dos pasos en comparación con una convolución estándar? Analice críticamente por qué, como director del proyecto, usted acepta pagar ese "costo" en este escenario comercial.

R//

En MobileNet, la Depthwise Separable Convolution divide una convolución tradicional en dos tareas distintas. Primero realiza el filtrado espacial con una convolución depthwise, donde cada canal de entrada se procesa por separado con su propio filtro. Así, cada mapa de características detecta bordes, texturas o patrones locales dentro de ese mismo canal, pero sin mezclar información con los demás. Después aplica una convolución pointwise de 1x1, cuya función ya no es buscar patrones espaciales, sino combinar los canales generados antes para construir representaciones más útiles para la clasificación. En términos simples, primero se analiza dónde están los patrones y luego se integra qué relación tienen entre sí los canales.

El precio de esta eficiencia es una pérdida de expresividad matemática frente a una convolución estándar. En una convolución normal, cada filtro aprende al mismo tiempo relaciones espaciales y relaciones entre canales dentro de una sola operación. Eso le da a la red una capacidad más rica para modelar interacciones complejas entre color, textura y forma desde etapas tempranas. En cambio, al separar ambos pasos, MobileNet impone una restricción estructural. Primero aprende patrones espaciales por canal de forma independiente y solo después mezcla los canales. Eso reduce la flexibilidad del espacio de funciones que la red puede aproximar, porque ya no todas las correlaciones espaciales entre canales se modelan de manera conjunta desde el inicio.

Como director del proyecto, acepto pagar ese costo porque en este escenario el criterio dominante no es alcanzar la máxima potencia teórica, sino lograr una solución usable, rápida y confiable en condiciones reales. El agricultor utilizará la aplicación en un teléfono Android de gama baja, en el campo y sin conexión a internet. En ese contexto, un modelo más pesado podría ofrecer una mejora marginal en precisión, pero sería inútil si tarda demasiado en responder, consume demasiada batería, ocupa demasiado almacenamiento o ni siquiera puede ejecutarse fluidamente en el dispositivo.

***Referencias:***

Szeliski, R. (2022). Computer Vision: Algorithms and Applications. Springer Nature.

Lin, M., Chen, Q., & Yan, S. (2013). Network In Network. CoRR, abs/1312.4400.

# Task 2

## Instalación y dependencias

In [2]:

```
import os
```

```python
import time
import copy
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models

from sklearn.metrics import f1_score, classification_report, confusion_matrix
import seaborn as sns

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Dispositivo: {device}")
```

```
Dispositivo: cuda
```

# 1. Descarga del dataset y división Train/Val/Test (70/15/15) con Data Augmentation

In [3]:

```python
import kagglehub

# Descargar el dataset de enfermedades de hojas de mango
path = kagglehub.dataset_download("aryashah2k/mango-leaf-disease-dataset")
print(f"Dataset descargado en: {path}")

# Buscar la carpeta raíz que contiene las subcarpetas de clases
for root, dirs, files in os.walk(path):
    if dirs and any(not d.startswith('.') for d in dirs):
        # Verificar si las subcarpetas contienen imágenes
        sample_dir = os.path.join(root, [d for d in dirs if not d.startswith('.')][0])
        if any(f.lower().endswith('.jpg', '.jpeg', '.png')) for f in os.listdir(sample_
dir)):
            DATA_ROOT = root
            break

print(f"Directorio raíz del dataset: {DATA_ROOT}")
print(f"Clases encontradas: {sorted(os.listdir(DATA_ROOT))}")
print(f"Número de clases: {len([d for d in os.listdir(DATA_ROOT) if os.path.isdir(os.path
.join(DATA_ROOT, d))])}")
```

```
/usr/lib/python3.14/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Plea
se update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_in
stall.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Dataset descargado en: /home/Japo/.cache/kagglehub/datasets/aryashah2k/mango-leaf-disease
-dataset/versions/1
Directorio raíz del dataset: /home/Japo/.cache/kagglehub/datasets/aryashah2k/mango-leaf-d
isease-dataset/versions/1
Clases encontradas: ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Ga
ll Midge', 'Healthy', 'Powdery Mildew', 'Sooty Mould']
Número de clases: 8
```

In [4]:

```python
# Definir transformaciones
# Para entrenamiento: Data Augmentation (rotaciones, flips, recortes)
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
```

```
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

# Para validación y prueba: solo resize y normalización
eval_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

# InceptionV3 requiere entrada de 299x299
train_transform_inception = transforms.Compose([
    transforms.Resize((320, 320)),
    transforms.RandomResizedCrop(299, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])

eval_transform_inception = transforms.Compose([
    transforms.Resize((320, 320)),
    transforms.CenterCrop(299),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225]),
])
```

In [5]:

```
# Cargar dataset completo (sin transformación aún, solo para obtener índices)
full_dataset = datasets.ImageFolder(DATA_ROOT)
class_names = full_dataset.classes
num_classes = len(class_names)

print(f"Total de imágenes: {len(full_dataset)}")
print(f"Clases ({num_classes}): {class_names}")

# Contar imágenes por clase
from collections import Counter
label_counts = Counter([label for _, label in full_dataset.samples])
for cls_idx, cls_name in enumerate(class_names):
    print(f"  {cls_name}: {label_counts[cls_idx]} imágenes")
```

```
Total de imágenes: 4000
Clases (8): ['Anthracnose', 'Bacterial Canker', 'Cutting Weevil', 'Die Back', 'Gall Midge
', 'Healthy', 'Powdery Mildew', 'Sooty Mould']
  Anthracnose: 500 imágenes
  Bacterial Canker: 500 imágenes
  Cutting Weevil: 500 imágenes
  Die Back: 500 imágenes
  Gall Midge: 500 imágenes
  Healthy: 500 imágenes
  Powdery Mildew: 500 imágenes
  Sooty Mould: 500 imágenes
```

In [6]:

```
# División estratificada: 70% train, 15% val, 15% test
from sklearn.model_selection import train_test_split

targets = [s[1] for s in full_dataset.samples]
indices = list(range(len(full_dataset)))

# Primera división: 70% train, 30% temp
```

```
train_idx, temp_idx = train_test_split(
    indices, test_size=0.30, random_state=42, stratify=targets
)

# Segunda división: 50/50 del 30% → 15% val, 15% test
temp_targets = [targets[i] for i in temp_idx]
val_idx, test_idx = train_test_split(
    temp_idx, test_size=0.50, random_state=42, stratify=temp_targets
)

print(f"Train: {len(train_idx)} | Val: {len(val_idx)} | Test: {len(test_idx)}")
print(f"Proporciones: {len(train_idx)/len(indices):.2f} / {len(val_idx)/len(indices):.2f}
/ {len(test_idx)/len(indices):.2f}")
```

```
Train: 2800 | Val: 600 | Test: 600
Proporciones: 0.70 / 0.15 / 0.15
```

In [7]:

```
def get_dataloaders(train_tf, eval_tf, batch_size=32):
    """Crea DataLoaders aplicando las transformaciones correspondientes."""
    # Datasets con transformaciones específicas
    train_ds = datasets.ImageFolder(DATA_ROOT, transform=train_tf)
    val_ds = datasets.ImageFolder(DATA_ROOT, transform=eval_tf)
    test_ds = datasets.ImageFolder(DATA_ROOT, transform=eval_tf)

    # Subsets con los índices de la división
    train_subset = Subset(train_ds, train_idx)
    val_subset = Subset(val_ds, val_idx)
    test_subset = Subset(test_ds, test_idx)

    train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True, num_wor
kers=2, pin_memory=True)
    val_loader = DataLoader(val_subset, batch_size=batch_size, shuffle=False, num_worker
s=2, pin_memory=True)
    test_loader = DataLoader(test_subset, batch_size=batch_size, shuffle=False, num_work
ers=2, pin_memory=True)

    return train_loader, val_loader, test_loader

# Verificar una muestra
train_loader, val_loader, test_loader = get_dataloaders(train_transform, eval_transform)
images, labels = next(iter(train_loader))
print(f"Batch shape: {images.shape}, Labels: {labels[:8]}")
```

```
Batch shape: torch.Size([32, 3, 224, 224]), Labels: tensor([2, 7, 6, 2, 6, 2, 6, 2])
```

## 2. Modelos pre-entrenados con Transfer Learning

**Se cargan ResNet50, InceptionV3 y MobileNetV2 pre-entrenados en ImageNet. Se congelan las capas base y se reemplaza únicamente el cabezal de clasificación para 8 clases.**

In [8]:

```
def build_resnet50(num_classes):
    """ResNet50 con capas base congeladas y nuevo cabezal."""
    model = models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
    # Congelar todas las capas base
    for param in model.parameters():
        param.requires_grad = False
    # Reemplazar el cabezal de clasificación
    in_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features, num_classes)
    )
    return model

def build_inceptionv3(num_classes):
    """InceptionV3 con capas base congeladas y nuevo cabezal."""
```

```
    model = models.inception_v3(weights=models.Inception_V3_Weights.IMAGENET1K_V1)
    # Congelar todas las capas base
    for param in model.parameters():
        param.requires_grad = False
    # Reemplazar cabezal principal
    in_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features, num_classes)
    )
    # Reemplazar cabezal auxiliar
    in_features_aux = model.AuxLogits.fc.in_features
    model.AuxLogits.fc = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features_aux, num_classes)
    )
    return model

def build_mobilenetv2(num_classes):
    """MobileNetV2 con capas base congeladas y nuevo cabezal."""
    model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.IMAGENET1K_V1)
    # Congelar todas las capas base
    for param in model.parameters():
        param.requires_grad = False
    # Reemplazar el clasificador
    in_features = model.classifier[1].in_features
    model.classifier = nn.Sequential(
        nn.Dropout(0.3),
        nn.Linear(in_features, num_classes)
    )
    return model

# Construir los 3 modelos
model_resnet = build_resnet50(num_classes).to(device)
model_inception = build_inceptionv3(num_classes).to(device)
model_mobilenet = build_mobilenetv2(num_classes).to(device)

# Verificar parámetros entrenables
for name, model in [("ResNet50", model_resnet), ("InceptionV3", model_inception), ("Mobi
leNetV2", model_mobilenet)]:
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    print(f"{name}: {trainable:,} entrenables / {total:,} totales ({100*trainable/total:.
1f}%)")
```

```
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /home/Japo/.c
ache/torch/hub/checkpoints/resnet50-0676ba61.pth
```

```
100%|██████████| 97.8M/97.8M [00:18<00:00, 5.69MB/s]
```

```
Downloading: "https://download.pytorch.org/models/inception_v3_google-0cc3c7bd.pth" to /h
ome/Japo/.cache/torch/hub/checkpoints/inception_v3_google-0cc3c7bd.pth
```

```
100%|██████████| 104M/104M [00:15<00:00, 7.18MB/s]
```

```
Downloading: "https://download.pytorch.org/models/mobilenet_v2-b0353104.pth" to /home/Jap
o/.cache/torch/hub/checkpoints/mobilenet_v2-b0353104.pth
```

```
100%|██████████| 13.6M/13.6M [00:04<00:00, 3.07MB/s]
```

```
ResNet50: 16,392 entrenables / 23,524,424 totales (0.1%)
InceptionV3: 22,544 entrenables / 24,365,808 totales (0.1%)
MobileNetV2: 10,248 entrenables / 2,234,120 totales (0.5%)
```

## 3. Entrenamiento con Cross-Entropy, Adam y Early Stopping

In [9]:

```
def train_model(model, train_loader, val_loader, model_name,
                max_epochs=20, patience=5, lr=1e-3):
    """
```

```
    Entrena un modelo con Cross-Entropy Loss, Adam optimizer y Early Stopping.
    Retorna el modelo con los mejores pesos y el historial de entrenamiento.
    """
    criterion = nn.CrossEntropyLoss()
    # Solo optimizar parámetros que requieren gradiente
    optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=lr)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', patience=2,
factor=0.5)

    history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
    best_val_loss = float('inf')
    best_model_wts = copy.deepcopy(model.state_dict())
    epochs_no_improve = 0

    for epoch in range(max_epochs):
        # --- Entrenamiento ---
        model.train()
        running_loss, correct, total = 0.0, 0, 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()

            # InceptionV3 retorna salida principal + auxiliar en modo train
            if model_name == "InceptionV3" and model.training:
                outputs, aux_outputs = model(images)
                loss = criterion(outputs, labels) + 0.4 * criterion(aux_outputs, labels)
            else:
                outputs = model(images)
                loss = criterion(outputs, labels)

            loss.backward()
            optimizer.step()

            running_loss += loss.item() * images.size(0)
            _, preds = torch.max(outputs, 1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

        train_loss = running_loss / total
        train_acc = correct / total

        # --- Validación ---
        model.eval()
        running_loss, correct, total = 0.0, 0, 0

        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                loss = criterion(outputs, labels)

                running_loss += loss.item() * images.size(0)
                _, preds = torch.max(outputs, 1)
                correct += (preds == labels).sum().item()
                total += labels.size(0)

        val_loss = running_loss / total
        val_acc = correct / total

        scheduler.step(val_loss)

        history['train_loss'].append(train_loss)
        history['val_loss'].append(val_loss)
        history['train_acc'].append(train_acc)
        history['val_acc'].append(val_acc)

        print(f"  Epoch {epoch+1}/{max_epochs} | "
              f"Train Loss: {train_loss:.4f} Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f} Acc: {val_acc:.4f}")

        # Early Stopping
```

```
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_model_wts = copy.deepcopy(model.state_dict())
            epochs_no_improve = 0
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                print(f"  Early stopping en epoch {epoch+1}")
                break

    model.load_state_dict(best_model_wts)
    return model, history
```

In [10]:

```
# ==================== Entrenar ResNet50 ====================
print("=" * 60)
print("Entrenando ResNet50...")
print("=" * 60)
train_loader, val_loader, test_loader = get_dataloaders(train_transform, eval_transform)
model_resnet, history_resnet = train_model(
    model_resnet, train_loader, val_loader, "ResNet50", max_epochs=20, patience=5
)
```

```
============================================================
Entrenando ResNet50...
============================================================
  Epoch 1/20 | Train Loss: 0.9805 Acc: 0.7389 | Val Loss: 0.3888 Acc: 0.9417
  Epoch 2/20 | Train Loss: 0.3484 Acc: 0.9386 | Val Loss: 0.2021 Acc: 0.9717
  Epoch 3/20 | Train Loss: 0.2600 Acc: 0.9404 | Val Loss: 0.1634 Acc: 0.9767
  Epoch 4/20 | Train Loss: 0.1984 Acc: 0.9550 | Val Loss: 0.1166 Acc: 0.9817
  Epoch 5/20 | Train Loss: 0.1837 Acc: 0.9557 | Val Loss: 0.1059 Acc: 0.9783
  Epoch 6/20 | Train Loss: 0.1417 Acc: 0.9664 | Val Loss: 0.0861 Acc: 0.9850
  Epoch 7/20 | Train Loss: 0.1322 Acc: 0.9671 | Val Loss: 0.1035 Acc: 0.9783
  Epoch 8/20 | Train Loss: 0.1334 Acc: 0.9593 | Val Loss: 0.0776 Acc: 0.9833
  Epoch 9/20 | Train Loss: 0.1264 Acc: 0.9671 | Val Loss: 0.0744 Acc: 0.9817
  Epoch 10/20 | Train Loss: 0.1203 Acc: 0.9668 | Val Loss: 0.0704 Acc: 0.9833
  Epoch 11/20 | Train Loss: 0.1072 Acc: 0.9704 | Val Loss: 0.0585 Acc: 0.9867
  Epoch 12/20 | Train Loss: 0.1136 Acc: 0.9636 | Val Loss: 0.0809 Acc: 0.9783
  Epoch 13/20 | Train Loss: 0.1095 Acc: 0.9675 | Val Loss: 0.0485 Acc: 0.9900
  Epoch 14/20 | Train Loss: 0.0950 Acc: 0.9721 | Val Loss: 0.0505 Acc: 0.9867
  Epoch 15/20 | Train Loss: 0.0966 Acc: 0.9696 | Val Loss: 0.0484 Acc: 0.9883
  Epoch 16/20 | Train Loss: 0.0918 Acc: 0.9725 | Val Loss: 0.0592 Acc: 0.9833
  Epoch 17/20 | Train Loss: 0.0974 Acc: 0.9689 | Val Loss: 0.0479 Acc: 0.9850
  Epoch 18/20 | Train Loss: 0.0848 Acc: 0.9704 | Val Loss: 0.0606 Acc: 0.9867
  Epoch 19/20 | Train Loss: 0.1066 Acc: 0.9632 | Val Loss: 0.0522 Acc: 0.9833
  Epoch 20/20 | Train Loss: 0.0824 Acc: 0.9729 | Val Loss: 0.0458 Acc: 0.9883
```

In [11]:

```
# ==================== Entrenar InceptionV3 ====================
print("=" * 60)
print("Entrenando InceptionV3...")
print("=" * 60)
train_loader_inc, val_loader_inc, test_loader_inc = get_dataloaders(
    train_transform_inception, eval_transform_inception
)
model_inception, history_inception = train_model(
    model_inception, train_loader_inc, val_loader_inc, "InceptionV3", max_epochs=20, pat
ience=5
)
```

```
============================================================
Entrenando InceptionV3...
============================================================
  Epoch 1/20 | Train Loss: 1.5943 Acc: 0.6175 | Val Loss: 1.0111 Acc: 0.8550
  Epoch 2/20 | Train Loss: 0.7224 Acc: 0.8654 | Val Loss: 0.6330 Acc: 0.9283
  Epoch 3/20 | Train Loss: 0.5533 Acc: 0.8786 | Val Loss: 0.5988 Acc: 0.8850
  Epoch 4/20 | Train Loss: 0.4370 Acc: 0.9039 | Val Loss: 0.4791 Acc: 0.9233
  Epoch 5/20 | Train Loss: 0.3961 Acc: 0.9136 | Val Loss: 0.5549 Acc: 0.8650
  Epoch 6/20 | Train Loss: 0.3751 Acc: 0.9075 | Val Loss: 0.4079 Acc: 0.9067
  Epoch 7/20 | Train Loss: 0.3441 Acc: 0.9107 | Val Loss: 0.3722 Acc: 0.9267
```

```
  Epoch 8/20 | Train Loss: 0.3079 Acc: 0.9218 | Val Loss: 0.2884 Acc: 0.9533
  Epoch 9/20 | Train Loss: 0.3233 Acc: 0.9182 | Val Loss: 0.3197 Acc: 0.9367
  Epoch 10/20 | Train Loss: 0.2905 Acc: 0.9207 | Val Loss: 0.3575 Acc: 0.9083
  Epoch 11/20 | Train Loss: 0.2841 Acc: 0.9196 | Val Loss: 0.3386 Acc: 0.9067
  Epoch 12/20 | Train Loss: 0.2844 Acc: 0.9218 | Val Loss: 0.2798 Acc: 0.9300
  Epoch 13/20 | Train Loss: 0.2655 Acc: 0.9293 | Val Loss: 0.2546 Acc: 0.9483
  Epoch 14/20 | Train Loss: 0.2518 Acc: 0.9343 | Val Loss: 0.2672 Acc: 0.9450
  Epoch 15/20 | Train Loss: 0.2484 Acc: 0.9325 | Val Loss: 0.2635 Acc: 0.9250
  Epoch 16/20 | Train Loss: 0.2756 Acc: 0.9232 | Val Loss: 0.2411 Acc: 0.9500
  Epoch 17/20 | Train Loss: 0.2547 Acc: 0.9325 | Val Loss: 0.2382 Acc: 0.9567
  Epoch 18/20 | Train Loss: 0.2477 Acc: 0.9375 | Val Loss: 0.2302 Acc: 0.9450
  Epoch 19/20 | Train Loss: 0.2632 Acc: 0.9250 | Val Loss: 0.2091 Acc: 0.9600
  Epoch 20/20 | Train Loss: 0.2656 Acc: 0.9268 | Val Loss: 0.2178 Acc: 0.9567
```

In [12]:

```
# ==================== Entrenar MobileNetV2 ====================
print("=" * 60)
print("Entrenando MobileNetV2...")
print("=" * 60)
train_loader, val_loader, test_loader = get_dataloaders(train_transform, eval_transform)
model_mobilenet, history_mobilenet = train_model(
    model_mobilenet, train_loader, val_loader, "MobileNetV2", max_epochs=20, patience=5
)
```

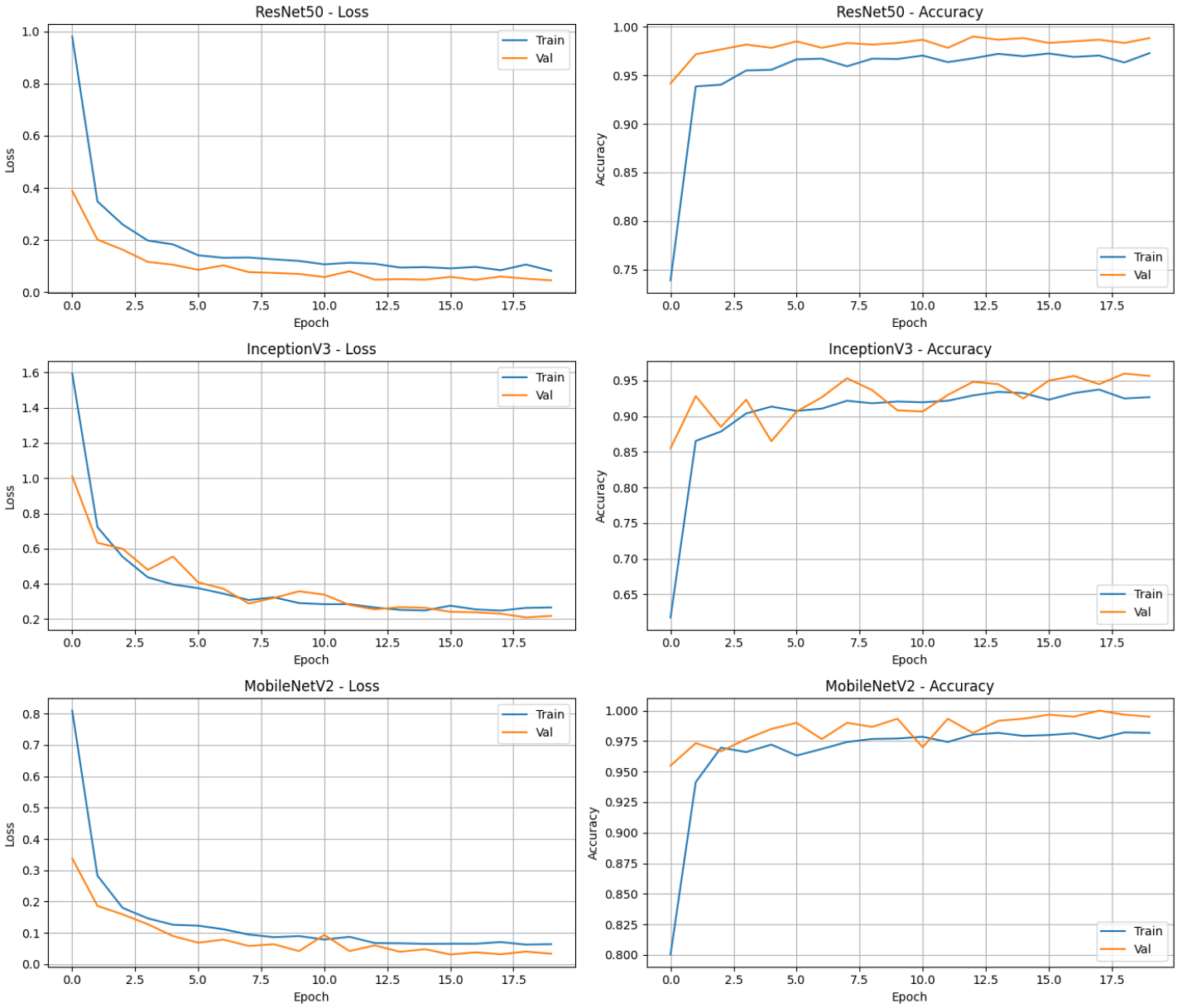
```
============================================================
Entrenando MobileNetV2...
============================================================
  Epoch 1/20 | Train Loss: 0.8101 Acc: 0.8004 | Val Loss: 0.3379 Acc: 0.9550
  Epoch 2/20 | Train Loss: 0.2828 Acc: 0.9414 | Val Loss: 0.1860 Acc: 0.9733
  Epoch 3/20 | Train Loss: 0.1797 Acc: 0.9696 | Val Loss: 0.1589 Acc: 0.9667
  Epoch 4/20 | Train Loss: 0.1462 Acc: 0.9661 | Val Loss: 0.1274 Acc: 0.9767
  Epoch 5/20 | Train Loss: 0.1258 Acc: 0.9721 | Val Loss: 0.0895 Acc: 0.9850
  Epoch 6/20 | Train Loss: 0.1228 Acc: 0.9632 | Val Loss: 0.0683 Acc: 0.9900
  Epoch 7/20 | Train Loss: 0.1113 Acc: 0.9686 | Val Loss: 0.0782 Acc: 0.9767
  Epoch 8/20 | Train Loss: 0.0945 Acc: 0.9743 | Val Loss: 0.0581 Acc: 0.9900
  Epoch 9/20 | Train Loss: 0.0859 Acc: 0.9768 | Val Loss: 0.0635 Acc: 0.9867
  Epoch 10/20 | Train Loss: 0.0895 Acc: 0.9771 | Val Loss: 0.0415 Acc: 0.9933
  Epoch 11/20 | Train Loss: 0.0785 Acc: 0.9786 | Val Loss: 0.0932 Acc: 0.9700
  Epoch 12/20 | Train Loss: 0.0872 Acc: 0.9743 | Val Loss: 0.0415 Acc: 0.9933
  Epoch 13/20 | Train Loss: 0.0673 Acc: 0.9804 | Val Loss: 0.0600 Acc: 0.9817
  Epoch 14/20 | Train Loss: 0.0666 Acc: 0.9818 | Val Loss: 0.0394 Acc: 0.9917
  Epoch 15/20 | Train Loss: 0.0647 Acc: 0.9793 | Val Loss: 0.0476 Acc: 0.9933
  Epoch 16/20 | Train Loss: 0.0651 Acc: 0.9800 | Val Loss: 0.0307 Acc: 0.9967
  Epoch 17/20 | Train Loss: 0.0651 Acc: 0.9814 | Val Loss: 0.0376 Acc: 0.9950
  Epoch 18/20 | Train Loss: 0.0704 Acc: 0.9771 | Val Loss: 0.0314 Acc: 1.0000
  Epoch 19/20 | Train Loss: 0.0624 Acc: 0.9821 | Val Loss: 0.0401 Acc: 0.9967
  Epoch 20/20 | Train Loss: 0.0637 Acc: 0.9818 | Val Loss: 0.0332 Acc: 0.9950
```

## Curvas de entrenamiento

In [13]:

```
fig, axes = plt.subplots(3, 2, figsize=(14, 12))

for i, (name, hist) in enumerate([
    ("ResNet50", history_resnet),
    ("InceptionV3", history_inception),
    ("MobileNetV2", history_mobilenet)
]):
    # Loss
    axes[i, 0].plot(hist['train_loss'], label='Train')
    axes[i, 0].plot(hist['val_loss'], label='Val')
    axes[i, 0].set_title(f'{name} - Loss')
    axes[i, 0].set_xlabel('Epoch')
    axes[i, 0].set_ylabel('Loss')
    axes[i, 0].legend()
    axes[i, 0].grid(True)

    # Accuracy
```

```
    axes[i, 1].plot(hist['train_acc'], label='Train')
    axes[i, 1].plot(hist['val_acc'], label='Val')
    axes[i, 1].set_title(f'{name} - Accuracy')
    axes[i, 1].set_xlabel('Epoch')
    axes[i, 1].set_ylabel('Accuracy')
    axes[i, 1].legend()
    axes[i, 1].grid(True)

plt.tight_layout()
plt.show()
```




# 4. Evaluación y métricas comparativas

**Para cada modelo se registra:**

- **Accuracy** en test
- **F1-Score (Macro)** en test
- **Tamaño del modelo** en MB (.pth)
- **Tiempo de inferencia** promedio por imagen (ms, sobre 100 imágenes)

In [14]:

```
def evaluate_model(model, test_loader, model_name, save_path):
    """Evalúa un modelo y retorna todas las métricas requeridas."""
    model.eval()
    all_preds = []
    all_labels = []
```

```python
    # --- Accuracy y F1-Score ---
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, preds = torch.max(outputs, 1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    accuracy = np.mean(np.array(all_preds) == np.array(all_labels))
    f1_macro = f1_score(all_labels, all_preds, average='macro')

    # --- Tamaño del modelo en disco ---
    torch.save(model.state_dict(), save_path)
    model_size_mb = os.path.getsize(save_path) / (1024 * 1024)

    # --- Tiempo de inferencia (promedio sobre 100 imágenes) ---
    # Obtener 100 imágenes del test set
    test_images = []
    for images, _ in test_loader:
        for img in images:
            test_images.append(img.unsqueeze(0).to(device))
            if len(test_images) >= 100:
                break
        if len(test_images) >= 100:
            break

    # Warmup
    with torch.no_grad():
        for _ in range(10):
            _ = model(test_images[0])

    # Medir tiempo
    if device.type == 'cuda':
        torch.cuda.synchronize()

    times = []
    with torch.no_grad():
        for img in test_images:
            if device.type == 'cuda':
                torch.cuda.synchronize()
            start = time.perf_counter()
            _ = model(img)
            if device.type == 'cuda':
                torch.cuda.synchronize()
            end = time.perf_counter()
            times.append((end - start) * 1000)  # ms

    avg_inference_ms = np.mean(times)

    return {
        'model_name': model_name,
        'accuracy': accuracy,
        'f1_macro': f1_macro,
        'model_size_mb': model_size_mb,
        'inference_ms': avg_inference_ms,
        'all_preds': all_preds,
        'all_labels': all_labels,
    }
```

In [15]:

```python
# Evaluar los 3 modelos
# DataLoaders para test (224x224 para ResNet y MobileNet, 299x299 para Inception)
_, _, test_loader_224 = get_dataloaders(train_transform, eval_transform)
_, _, test_loader_299 = get_dataloaders(train_transform_inception, eval_transform_incept
ion)

results = {}

print("Evaluando ResNet50...")
```

```
results['ResNet50'] = evaluate_model(model_resnet, test_loader_224, "ResNet50", "resnet50_mango.pth")

print("Evaluando InceptionV3...")
results['InceptionV3'] = evaluate_model(model_inception, test_loader_299, "InceptionV3", "inceptionv3_mango.pth")

print("Evaluando MobileNetV2...")
results['MobileNetV2'] = evaluate_model(model_mobilenet, test_loader_224, "MobileNetV2", "mobilenetv2_mango.pth")

print("\nEvaluación completa.")
```

```
Evaluando ResNet50...
Evaluando InceptionV3...
Evaluando MobileNetV2...

Evaluación completa.
```

In [16]:

```
# Tabla comparativa de métricas
import pandas as pd

summary = []
for name in ['ResNet50', 'InceptionV3', 'MobileNetV2']:
    r = results[name]
    summary.append({
        'Modelo': name,
        'Accuracy (%)': f"{r['accuracy']*100:.2f}",
        'F1-Score (Macro)': f"{r['f1_macro']:.4f}",
        'Tamaño (MB)': f"{r['model_size_mb']:.2f}",
        'Inferencia (ms)': f"{r['inference_ms']:.2f}",
    })

df_summary = pd.DataFrame(summary)
print("=" * 70)
print("TABLA COMPARATIVA DE MÉTRICAS")
print("=" * 70)
display(df_summary)
```

```
======================================================================
TABLA COMPARATIVA DE MÉTRICAS
======================================================================
```

| | Modelo | Accuracy (%) | F1-Score (Macro) | Tamaño (MB) | Inferencia (ms) |
|---|---|---|---|---|---|
| 0 | ResNet50 | 98.50 | 0.9849 | 90.04 | 2.74 |
| 1 | InceptionV3 | 96.33 | 0.9632 | 93.28 | 4.10 |
| 2 | MobileNetV2 | 99.50 | 0.9950 | 8.76 | 2.03 |

## Classification Report y Matrices de Confusión

In [17]:

```
# Classification reports
for name in ['ResNet50', 'InceptionV3', 'MobileNetV2']:
    r = results[name]
    print(f"\n{'='*60}")
    print(f"Classification Report - {name}")
    print(f"{'='*60}")
    print(classification_report(r['all_labels'], r['all_preds'], target_names=class_names))
```

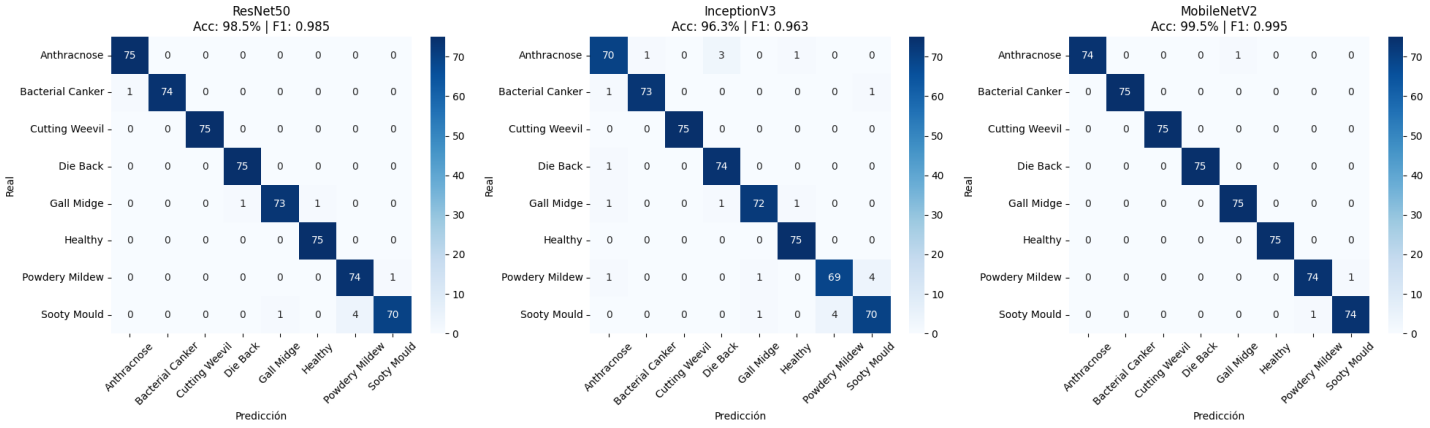
```
============================================================
Classification Report - ResNet50
============================================================
              precision    recall  f1-score   support
```

```
     Anthracnose       0.99      1.00      0.99        75
Bacterial Canker       1.00      0.99      0.99        75
  Cutting Weevil       1.00      1.00      1.00        75
        Die Back       0.99      1.00      0.99        75
      Gall Midge       0.99      0.97      0.98        75
         Healthy       0.99      1.00      0.99        75
  Powdery Mildew       0.95      0.99      0.97        75
     Sooty Mould       0.99      0.93      0.96        75

        accuracy                           0.98       600
       macro avg       0.99      0.99      0.98       600
    weighted avg       0.99      0.98      0.98       600


============================================================
Classification Report - InceptionV3
============================================================
                   precision    recall  f1-score   support

     Anthracnose       0.95      0.93      0.94        75
Bacterial Canker       0.99      0.97      0.98        75
  Cutting Weevil       1.00      1.00      1.00        75
        Die Back       0.95      0.99      0.97        75
      Gall Midge       0.97      0.96      0.97        75
         Healthy       0.97      1.00      0.99        75
  Powdery Mildew       0.95      0.92      0.93        75
     Sooty Mould       0.93      0.93      0.93        75

        accuracy                           0.96       600
       macro avg       0.96      0.96      0.96       600
    weighted avg       0.96      0.96      0.96       600


============================================================
Classification Report - MobileNetV2
============================================================
                   precision    recall  f1-score   support

     Anthracnose       1.00      0.99      0.99        75
Bacterial Canker       1.00      1.00      1.00        75
  Cutting Weevil       1.00      1.00      1.00        75
        Die Back       1.00      1.00      1.00        75
      Gall Midge       0.99      1.00      0.99        75
         Healthy       1.00      1.00      1.00        75
  Powdery Mildew       0.99      0.99      0.99        75
     Sooty Mould       0.99      0.99      0.99        75

        accuracy                           0.99       600
       macro avg       1.00      1.00      0.99       600
    weighted avg       1.00      0.99      0.99       600
```

In [18]:

```python
# Matrices de confusión
fig, axes = plt.subplots(1, 3, figsize=(20, 6))

for i, name in enumerate(['ResNet50', 'InceptionV3', 'MobileNetV2']):
    r = results[name]
    cm = confusion_matrix(r['all_labels'], r['all_preds'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names, ax=axes[i])
    axes[i].set_title(f'{name}\nAcc: {r["accuracy"]*100:.1f}% | F1: {r["f1_macro"]:.3f}'
)
    axes[i].set_ylabel('Real')
    axes[i].set_xlabel('Predicción')
    axes[i].tick_params(axis='x', rotation=45)
    axes[i].tick_params(axis='y', rotation=0)

plt.tight_layout()
```

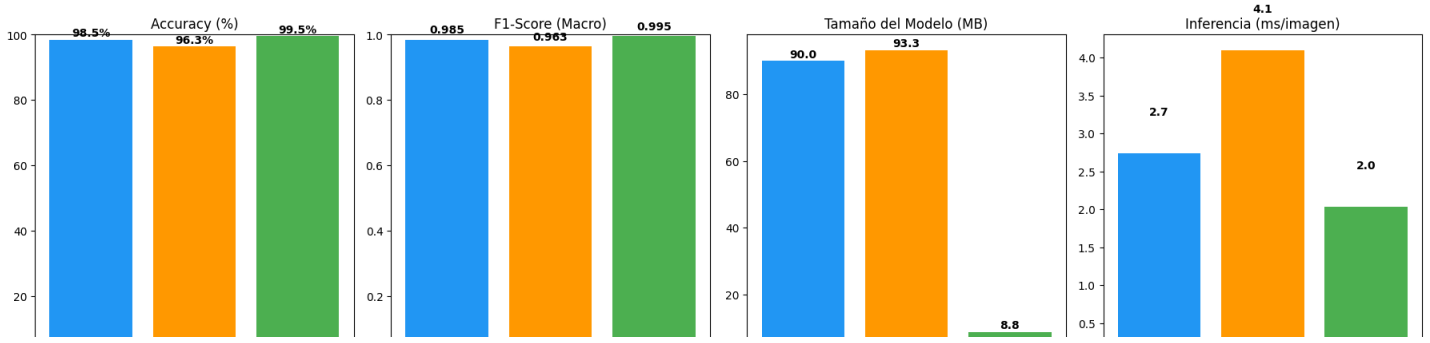
```
plt.show()
```




## Gráficas comparativas de métricas

In [19]:

```
# Gráficas de barras comparativas
model_names = ['ResNet50', 'InceptionV3', 'MobileNetV2']
colors = ['#2196F3', '#FF9800', '#4CAF50']

fig, axes = plt.subplots(1, 4, figsize=(18, 5))

# Accuracy
accs = [results[m]['accuracy'] * 100 for m in model_names]
axes[0].bar(model_names, accs, color=colors)
axes[0].set_title('Accuracy (%)')
axes[0].set_ylim(0, 100)
for j, v in enumerate(accs):
    axes[0].text(j, v + 1, f'{v:.1f}%', ha='center', fontweight='bold')

# F1-Score
f1s = [results[m]['f1_macro'] for m in model_names]
axes[1].bar(model_names, f1s, color=colors)
axes[1].set_title('F1-Score (Macro)')
axes[1].set_ylim(0, 1)
for j, v in enumerate(f1s):
    axes[1].text(j, v + 0.02, f'{v:.3f}', ha='center', fontweight='bold')

# Tamaño del modelo
sizes = [results[m]['model_size_mb'] for m in model_names]
axes[2].bar(model_names, sizes, color=colors)
axes[2].set_title('Tamaño del Modelo (MB)')
for j, v in enumerate(sizes):
    axes[2].text(j, v + 1, f'{v:.1f}', ha='center', fontweight='bold')

# Tiempo de inferencia
times = [results[m]['inference_ms'] for m in model_names]
axes[3].bar(model_names, times, color=colors)
axes[3].set_title('Inferencia (ms/imagen)')
for j, v in enumerate(times):
    axes[3].text(j, v + 0.5, f'{v:.1f}', ha='center', fontweight='bold')

plt.tight_layout()
plt.show()
```



# Task 3

Con base en las metricas obtenidas en el Task 2, la recomendacion ejecutiva es mandar a produccion MobileNetV2. En este laboratorio no hubo un sacrificio de precision por escoger la arquitectura mas ligera; al contrario, MobileNetV2 obtuvo el mejor desempeno global en las cuatro metricas solicitadas y, ademas, es el modelo mas alineado con la restriccion de negocio: inferencia 100% offline en telefonos Android de gama baja usados por agricultores guatemaltecos en campo.

## Tabla comparativa final

| Modelo | Accuracy (%) | F1-Score (Macro) | Tamano (MB) | Inferencia (ms/imagen) |
|---|---|---|---|---|
| ResNet50 | 98.50 | 0.9849 | 90.04 | 2.74 |
| InceptionV3 | 96.33 | 0.9632 | 93.28 | 4.10 |
| MobileNetV2 | 99.50 | 0.9950 | 8.76 | 2.03 |

## Comparacion de MobileNetV2 frente a ResNet50 e InceptionV3

El primer hallazgo importante es que MobileNetV2 no sacrifico accuracy. Frente a ResNet50, MobileNetV2 mejoro la exactitud en +1.00 punto porcentual (99.50% vs 98.50%) y aumento el F1 macro en +0.0101. Frente a InceptionV3, la mejora fue todavia mas clara: +3.17 puntos porcentuales de accuracy (99.50% vs 96.33%) y +0.0318 en F1 macro. Desde la perspectiva del cliente, esto significa que el modelo mas pequeno no solo es mas barato de desplegar, sino tambien el que menos se equivoca en el conjunto de prueba.

El segundo hallazgo es la diferencia extrema en tamano. MobileNetV2 ocupa 8.76 MB, mientras que ResNet50 ocupa 90.04 MB e InceptionV3 93.28 MB. En otras palabras, MobileNetV2 es aproximadamente 90.27% mas pequeño que ResNet50 **y 90.61% mas pequeño que InceptionV3**. Esta relacion si se correlaciona directamente con la arquitectura matematica descrita en el Task 1. ResNet mantiene convoluciones densas y una red profunda con muchos parametros, mientras que Inception agrega ramas paralelas y mas operaciones por bloque. En cambio, MobileNetV2 usa depthwise separable convolution, separando el filtrado espacial de la combinacion entre canales. Esa descomposicion reduce de forma drastica la cantidad de pesos y operaciones, por lo que el archivo final en disco es mucho menor y la carga sobre memoria RAM tambien disminuye.

El tercer hallazgo es el tiempo de inferencia. MobileNetV2 promedia 2.03 ms por imagen, contra 2.74 ms de ResNet50 y 4.10 ms de InceptionV3. Esto vuelve a dejarlo como el modelo dominante: es 25.91% mas rapido que ResNet50 y 50.49% mas rapido que InceptionV3. En un escenario de Edge AI, estos milisegundos importan porque se traducen en una experiencia de usuario mas fluida, menor consumo energetico y menos riesgo de que la aplicacion se sienta lenta en un telefono con CPU y memoria limitadas.

## Respuesta al CEO:

Considerando que los agricultores guatemaltecos usarán teléfonos con 2GB de RAM sin internet, ¿qué modelo exacto mandamos a producción y por qué? (Justifique por qué el modelo ganador es viable para Edge AI frente a los perdedores).

Si el objetivo es desplegar una aplicacion que funcione sin internet en telefonos Android con 2 GB de RAM, la decision correcta es MobileNetV2. La razon no es solo tecnica, sino comercial. El agricultor necesita una prediccion rapida, confiable y local; no necesita un modelo academicamente pesado que consuma mas almacenamiento, mas RAM y mas bateria sin ofrecer una mejora real en desempeno.

ResNet50 queda como un segundo lugar razonable desde precision, pero pierde en viabilidad operativa: pesa mas de 90 MB y aun asi queda por debajo de MobileNetV2 en accuracy, F1 e inferencia. InceptionV3 queda descartado con mas claridad porque combina lo peor de ambos mundos en este experimento: menor precision, mayor tamano y el tiempo de inferencia mas alto. Dicho de forma ejecutiva, ResNet50 e InceptionV3 son modelos validos para laboratorio o servidores con mas recursos, pero no son la mejor decision de produccion para este caso de uso rural y offline.

**Por lo tanto, el modelo que debe enviarse a produccion es MobileNetV2, porque ofrece la mejor combinacion entre precision, F1, ligereza y velocidad. Esta arquitectura cumple el criterio de Edge AI del proyecto: cabe con holgura en el dispositivo, reduce presion sobre memoria, acelera la inferencia local y mantiene la mejor calidad predictiva observada. Como paso posterior al laboratorio, la recomendacion tecnica seria exportarlo a un formato optimizado para movil y validarlo en hardware Android real, pero con la evidencia actual la eleccion para produccion es clara y defendible: MobileNetV2.**